

Materialized View in Amazon Redshift

To Begin with the Lab

Summary of the Lab

In this lab, a Materialized View named `tickets_mv` was created in **Amazon Redshift** using sales and event data from the Tikit sample database. A query joining the sales and event tables was executed first to analyze event-wise total sales. The query was then converted into a Materialized View to cache results and improve analytics performance for repetitive queries and dashboards. Base table data was later updated to demonstrate that Materialized Views use cached data until explicitly refreshed. Finally, the Materialized View was refreshed manually and validated successfully by confirming that the updated sales values were reflected in the query output.

- Navigate to **Amazon Redshift** → open **Redshift Query Editor v2**
- Create a schema for the lab data
- Connect to the Redshift cluster and select the dev database
- Create a new query editor tab for the lab

```
CREATE SCHEMA mat_view_demo;
```

- Set the schema search path

```
SET search_path TO mat_view_demo, public;
```

- Run the base query to calculate event-wise total sales

materialized view demo

Run all Isolated session redshift-clust... dev Last saved: 2 minutes ago

```

8
9 SELECT e.eventname eventname,
10 sum(s.pricepaid) total_sales
11 FROM event e, sales s
12 WHERE e.eventid = s.eventid
13 GROUP BY e.eventname
14 order by e.eventname;
15
16 -- create materialized view
17 CREATE MATERIALIZED VIEW tickets_mv
18 AS (SELECT e.eventname eventname,
19 sum(s.pricepaid) total_sales
20 FROM event e, sales s
21 WHERE e.eventid = s.eventid
22 GROUP BY e.eventname);
23
24 -- check the view data
25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');

```

Row 9, Col 1, Chr 832

Result 1 (100)

eventname	total_sales
.38 Special	134386
3 Doors Down	125816

Export Chart

```

SELECT e.eventname eventname,
       SUM(s.pricepaid) total_sales
FROM event e, sales s
WHERE e.eventid = s.eventid
GROUP BY e.eventname
ORDER BY e.eventname;

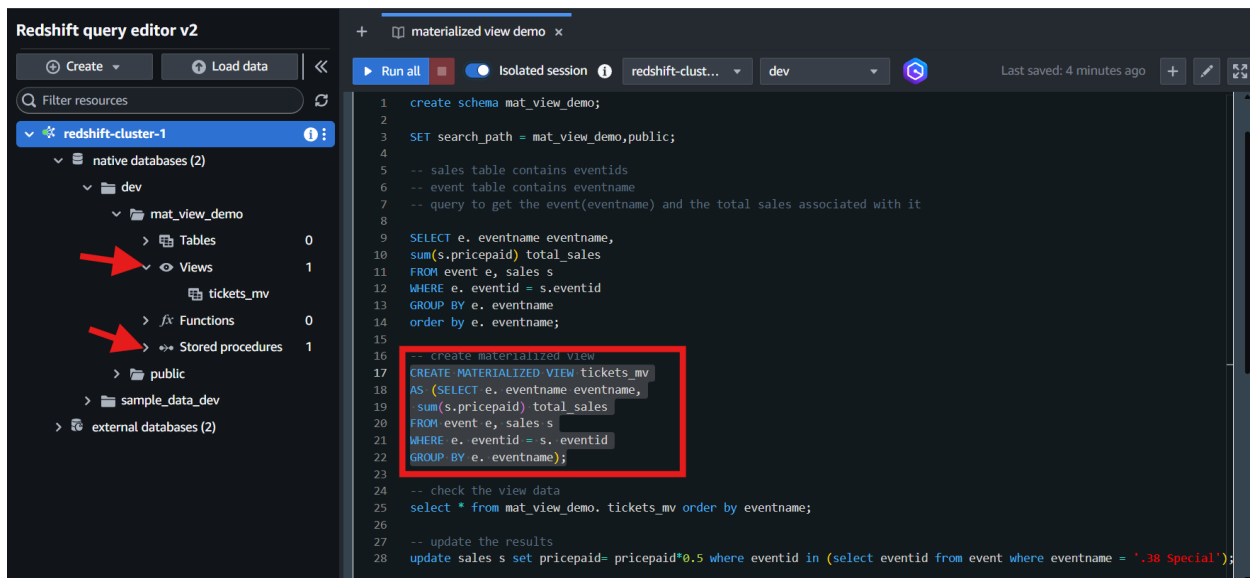
```

- Verify that the query returns 100 rows
- Create the Materialized View using the same query

```

CREATE MATERIALIZED VIEW tickets_mv AS
SELECT e.eventname eventname,
       SUM(s.pricepaid) total_sales
FROM event e, sales s
WHERE e.eventid = s.eventid
GROUP BY e.eventname;

```

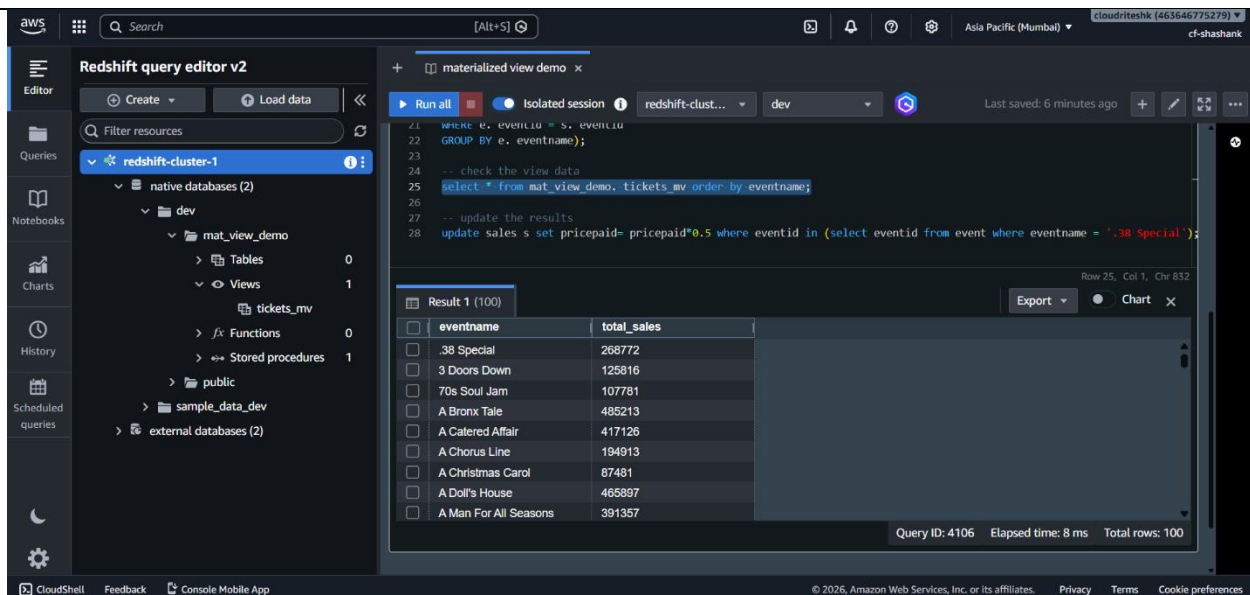


- Refresh the left navigation panel inside **Redshift Query Editor v2**
- Verify that the mat_view_demo schema now contains:
 - One View
 - One Internal Table
- Query the Materialized View

```

SELECT *
FROM mat_view_demo.tickets_mv
ORDER BY eventname;

```



- Verify that the Materialized View returns the same 100 rows as the original query
- Update the sales table data for the 0.38 Special event

```

UPDATE sales
SET pricepaid = pricepaid * 0.5
WHERE eventid IN (
    SELECT eventid
    FROM event
    WHERE eventname = '0.38 Special' );

```

The screenshot shows the Redshift query editor v2 interface. The left sidebar displays the database structure for 'redshift-cluster-1', including 'mat_view_demo' and 'tickets_mv'. The main editor area contains a SQL script with the following lines highlighted in a red box:

```

27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
29
30 -- refresh the view data
31 refresh materialized view mat_view_demo.tickets_mv;

```

Below the script, the 'Result 1' summary is displayed, also highlighted in a red box:

```

Summary
Affected rows: 426
Query ID: 4123
Elapsed time: 775ms
Result set query:
/* RDEV2-tytuHwVA */
update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special')

```

- Verify that approximately 426 rows were updated
- Query the Materialized View again

```

SELECT *
FROM mat_view_demo.tickets_mv
ORDER BY eventname;

```

The screenshot shows the Redshift query editor v2 interface. The left sidebar displays the database structure for 'redshift-cluster-1', including 'mat_view_demo' and 'tickets_mv'. The main editor area contains a SQL script with the following lines highlighted in a red box:

```

25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
29
30 -- refresh the view data
31 refresh materialized view mat_view_demo.tickets_mv;

```

Below the script, the 'Result 1 (100)' table is displayed, also highlighted in a red box:

eventname	total_sales
.38 Special	268772
3 Doors Down	125816
70s Soul Jam	107781
A Bronx Tale	485213
A Catered Affair	417126
A Chorus Line	194913
A Christmas Carol	87481
A Doll's House	465897
A Man For All Seasons	391357

At the bottom of the interface, the status bar shows: Query ID: 4164 Elapsed time: 6 ms Total rows: 100

- Observe that old cached values are still displayed
- Refresh the Materialized View manually

REFRESH MATERIALIZED VIEW mat_view_demo.tickets_mv;

The screenshot shows the Redshift query editor interface. On the left, the resource tree is expanded to show the 'dev' database, 'mat_view_demo' schema, and the 'tickets_mv' materialized view. The main query editor displays the following SQL script:

```

23
24 -- check the view data
25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
29
30 -- refresh the view data
31 refresh materialized view mat_view_demo.tickets_mv;
  
```

The 'refresh materialized view' statement is highlighted with a red box. Below the query editor, the 'Result 1' summary is displayed, showing an information message: 'Materialized view tickets_mv was incrementally updated successfully.' The summary also indicates 'Returned rows: 0', 'Query ID: 4193', 'Elapsed time: 3.6s', and 'Result set query:'.

- Query the Materialized View after refresh

SELECT *
FROM mat_view_demo.tickets_mv
ORDER BY eventname;

The screenshot shows the Redshift query editor interface with the same resource tree as the previous image. The main query editor displays the following SQL script:

```

23
24 -- check the view data
25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
29
30 -- refresh the view data
31 refresh materialized view mat_view_demo.tickets_mv;
  
```

The 'select * from mat_view_demo.tickets_mv order by eventname;' statement is highlighted with a red box. Below the query editor, the 'Result 1 (100)' table is displayed, showing the updated data. The table has two columns: 'eventname' and 'total_sales'. The data is as follows:

eventname	total_sales
.38 Special	67193
3 Doors Down	125816
70s Soul Jam	107781
A Bronx Tale	485213
A Catered Affair	417126
A Chorus Line	194913
A Christmas Carol	87481
A Doll's House	465897
A Man For All Seasons	391357

The table is highlighted with a red box. At the bottom right, the status bar shows 'Query ID: 4206', 'Elapsed time: 166 ms', and 'Total rows: 100'.

- Verify that the 0.38 Special event now shows updated sales values of approximately 0.5+ million
- Confirm that the Materialized View refresh completed successfully and reflected the latest base table data